

Embedded Systems Architectures - Laboratory Guide 9

IP protocols for IoT: MQTT

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Familiarization with the MQTT protocol
- Publishing information on MQTT topics: over TCP and over TLS
- Installation and configuration of an MQTT TLS broker

2. Introduction

The evolution of the Internet of Things (IoT) has motivated the search for efficient, reliable, and low-power communication solutions, especially in embedded systems, which operate with limited hardware, connectivity, and power resources. In this scenario, the Message Queuing Telemetry Transport (MQTT) protocol has become one of the main choices for real-time communication between connected devices.

MQTT is a lightweight messaging protocol based on the publisher-subscriber model, originally developed by IBM for industrial applications over low-bandwidth and high-latency networks. These characteristics make it especially suitable for embedded devices, such as the ESP32 family of microcontrollers, which frequently operate in environments with limited or unstable connectivity, such as home or rural Wi-Fi networks.

One of the main advantages of MQTT is its low communication overhead. Unlike heavier protocols like HTTP, MQTT sends compact messages, optimized for minimal data and power consumption, which is a crucial factor for battery-powered devices.

Furthermore, the protocol offers configurable Quality of Service (QoS) levels, ensuring message delivery even in the case of temporary connection loss.

The publisher-subscriber model adopted by MQTT also provides strong decoupling between message senders and receivers. This enables a more scalable, modular architecture in which sensors, actuators, servers, and mobile applications can interact asynchronously without direct dependencies on one another. Another important benefit is support for features such as message retention and last will, which make the system more robust and fault-tolerant.

2.1 MQTT Publisher-Subscriber Architecture

Figure 1 illustrates the publisher-subscriber architecture of MQTT. As can be observed, an architecture is adopted where communication between devices is mediated by a central element called a broker. The broker is responsible for receiving client messages and distributing them only to subscribers of the corresponding topic, ensuring an efficient, decoupled data flow.

For example, in a system where ESP32 devices monitor temperature, a client can post values to the sensors/temperature topic, while other clients subscribed to that topic receive updates automatically. This approach simplifies communication, avoids direct connections between devices, and enables greater scalability and flexibility, making it especially advantageous in embedded systems and IoT applications.

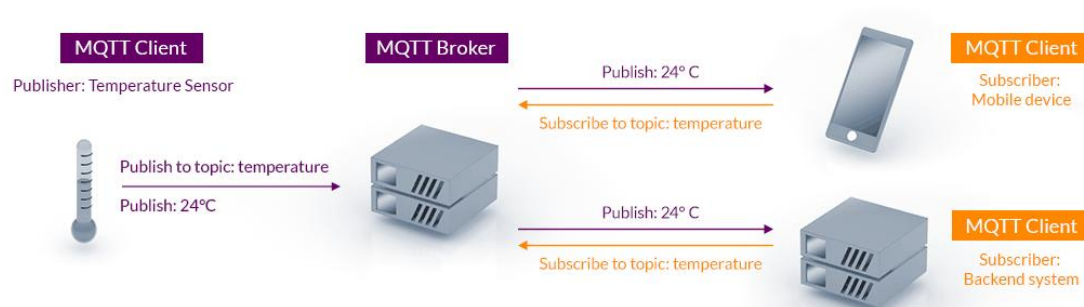


Figure 1 - MQTT publisher/subscriber architecture.

The ESP MQTT API supported by the ESP-IDF framework is an implementation of the MQTT protocol client, currently at version 5.0.

2.2 Quality of Service

The MQTT protocol defines three Quality of Service (QoS) levels that govern the delivery guarantees between clients and the broker. These levels are especially relevant in embedded systems, where network stability can vary and the balance between reliability and efficiency is essential:

- QoS 0 – “At most once”: The message is sent only once, without confirmation of receipt. It is the fastest and lightest mode, but it does not guarantee that the message will reach its destination.
- QoS 1 – “At least once”: The message is guaranteed to be delivered, but may be received more than once in case of retransmissions.
- QoS 2 – “Exactly once”: Ensures that the message is delivered only once, without duplication. It is the most secure level, but also the most resource-intensive and latency-intensive.

The choice of QoS level depends on the type of application: sensors with frequent updates can use QoS 0 for efficiency, while critical commands may require QoS 1 or QoS 2 for greater reliability. QoS is defined per message sent or per topic subscription.

2.3 Topic

In the MQTT protocol, a topic is a hierarchical string used to organize and classify messages that are published and subscribed to. It functions as a "channel" for sending (publishing) or receiving (subscribing) data. Topics are fundamental to the publisher/subscriber model, allowing devices to communicate in a decoupled way, that is, without the sender and receiver knowing each other directly:

- A publishing client sends messages to a specific topic.
- A subscribing client receives messages when it subscribes to that topic.

The structure of a topic is similar to a directory path, separated by slashes (/):

- Example: **house/room/temperature**

Each level represents a category or group of information, allowing the organization of the data in a logical and scalable way.

2.4 How to Use MQTT on the ESP32-C6

The MQTT protocol can be used on the ESP32-C6 through different transport modes, adapting to various security contexts, network infrastructure, and application requirements:

- MQTT over TCP (mqtt/tcp): The most basic form, using port 1883 and offering no encryption layer. This configuration is only suitable for controlled and reliable environments, such as local area networks (LANs) or VPNs, where security is guaranteed by other means. It is useful for testing or prototyping, allowing connection to a broker, simple topic publishing and subscription, and reconnection management in case of Wi-Fi failures. However, because it transmits credentials and data in plain text, it should not be used on open networks or the Internet.
- MQTT over TLS (mqtt/ssl): Recommended mode for production applications or those with Internet exposure and operates on port 8883. This mode adds end-to-end encryption via digital certificates, ensuring data confidentiality and integrity. On the ESP32-C6, it is possible to embed Certificate Authority (CA) certificates in the firmware to validate the server certificate and optionally support mutual authentication with a client certificate. This model is widely adopted in cloud-based solutions, such as AWS IoT or Azure, although it requires additional care with the certificate lifecycle, memory consumption, and reconnection and backoff policies.
- TLS with pre-shared keys (mqtt/ssl_psk): A lighter variation that replaces the certificates with a common symmetric key between the client and the broker. This approach reduces cryptographic complexity and memory usage, making

it useful in closed environments, such as laboratories or private industrial networks. However, secure management of shared keys is complex on a large scale, and many commercial brokers do not support this method, making it impractical for public applications.

- MQTT over WebSocket (mqtt/ws): An alternative that encapsulates MQTT traffic within an HTTP connection via WebSocket, typically on port 80. This approach is useful when the network infrastructure blocks direct TCP connections but allows HTTP traffic, such as in corporate networks with restrictive policies. However, when used without TLS (ws://), communication remains vulnerable. Therefore, in real-world scenarios, the secure version with TLS (mqtt/wss) is recommended, which uses port 443 and combines the advantages of WebSocket with TLS security.
- Secure MQTT over WebSocket with TLS (mqtt/wss): Effective in business contexts or public networks, where it is common for only the HTTPS port to be accessible. In addition to ensuring secure communication, it allows the MQTT broker to operate behind load balancers, HTTP proxies, or CDNs, sharing infrastructure with other web services. This configuration is ideal for IoT devices operating in environments with restrictive firewalls, and is often adopted as the standard solution in modern IoT architectures.

2.5 MQTT Broker Options

There are several MQTT brokers available for local or server installation, each with specific characteristics that adapt to different usage scenarios. One of the most popular is Eclipse Mosquitto, a lightweight, open-source solution maintained by the Eclipse Foundation. It is ideal for local testing, ESP32 applications, and small to medium-scale projects. It supports MQTT versions 3.1, 3.1.1, and 5.0, as well as features such as user and password authentication, TLS encryption, and message persistence. Its installation is simple, with packages available for Windows, Linux, macOS, and also via Docker.

For larger-scale applications and corporate environments, EMQX stands out as one of the most complete options. This broker is highly scalable, supporting millions of simultaneous connections, and includes a wide range of features, such as advanced authentication (including OAuth2 and JWT), access control (ACL), MQTT v5 support, integration with external databases and protocols, as well as a web interface for monitoring and management. It can be installed locally or in Docker and Kubernetes containers, and is also available in a cloud version.

Another robust alternative is HiveMQ CE (Community Edition), the free and open-source version of the HiveMQ broker. It focuses on reliability, performance, and integration with business systems. It supports MQTT v3.1.1 and v5, and is extensible through Java plugins, allowing it to easily adapt to different architectures. Installation is also cross-platform, with support for modern operating systems and deployment via Docker.

Finally, RabbitMQ, widely known as an AMQP broker, can also serve as an MQTT broker by enabling the MQTT plugin. This functionality makes RabbitMQ particularly useful in hybrid environments, where you want to integrate MQTT devices with existing AMQP-based systems. With support for TLS, authentication, administrative dashboards, and the ability to interconnect multiple protocols, RabbitMQ with MQTT support is ideal for those already using this technology and wanting to expand to IoT applications in an integrated way.

3. Practical Work

The practical work in this lesson is based on the ESP MQTT Reference API and the ESP32-C6 Wi-Fi Reference API, along with the ESP32-C6 Technical Reference Manual https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

3.1 Simple MQTT over TCP

The simple MQTT over TCP example demonstrates a basic MQTT communication application using an ESP32-C6, connecting to a broker on the standard port 1883 without encryption (i.e., without TLS). This example shows how to establish a

connection with the broker, configure essential parameters (e.g., URI, Client ID, topic), and implement the fundamental protocol operations: publishing messages, subscribing to topics, and managing automatic reconnections in case of network failures, such as Wi-Fi loss. The configuration is done via **menuconfig**, allowing the user to easily define the broker's URI and other parameters without altering the code. This example is especially useful for test environments or secure internal networks, since it does not use encryption; it is not recommended for use on public networks. The program starts the network subsystem, attempts to connect to the network defined in **menuconfig**, and logs events to track the connection process, including reconnection attempts and failures. When the connection is successful, the device obtains an IPv4 address via DHCP from the AP, and this information is displayed on the terminal.

Question 1: In VSCode, create a new project from the *protocols -> mqtt -> tcp* template. Using the **menuconfig** command, define the SSID and password of the Wi-Fi network you have configured on your phone. Access to this configuration in the project is found in the Example Connection Configuration section of **menuconfig**. Additionally, define the Broker URI as *mqtt://test.mosquitto.org:1883*. This configuration is available in the Example Configuration section of **menuconfig**. Compile the program and download the firmware for the SoC.

At this stage, you must ensure that your Access Point is correctly configured to allow Internet access so that the program can reach the test broker (*mqtt://test.mosquitto.org:1883*). Once the board initializes, you should see an event log in the Monitor window similar to the following:

```
...
I (23) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (291) cpu_start: Unicom app
...
I (405) main_task: Started on CPU0
I (405) main_task: Calling app_main()
...
I (425) example_connect: Start example_connect.
...
```

```

I (18235) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 6a:9b:ac:b5:55:f8
I (18245) wifi:security: WPA2-PSK, phy:ax, rssi:-50, cipher(pairwise:0x3, group:0x3), pmf:0,
...
I (19425) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (19535) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw:
172.20.10.1
I (19535) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (19535) example_common: Connected to example_netif_sta
I (19545) example_common: - IPv4 address: 172.20.10.2,
I (19545) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
ESP_IP6_ADDR_IS_LINK_LOCAL
I (19555) mqtt_example: Other event id:7
I (19565) main_task: Returned from app_main()
W (20685) wifi:<ba-add>idx:1, ifx:0, tid:0, TAHI:0x100f855, TALO:0xb5ac9b6a, (ssn:2, win:64,
cur_ssn:2),
CONF:0xc0000005
I (21385) mqtt_example: MQTT_EVENT_CONNECTED
I (21385) mqtt_example: sent publish successful, msg_id=36322
I (21385) mqtt_example: sent subscribe successful, msg_id=40368
I (21385) mqtt_example: sent subscribe successful, msg_id=8613
I (21395) mqtt_example: sent unsubscribe successful, msg_id=55753
I (21695) mqtt_example: MQTT_EVENT_PUBLISHED, msg_id=36322
I (21995) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=40368
I (21995) mqtt_example: sent publish successful, msg_id=0
I (22005) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=8613
I (22005) mqtt_example: sent publish successful, msg_id=0
I (22005) mqtt_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=55753
I (22405) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (22615) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data

```

Question 2: Analyze this result and the application's source code, consulting the API documentation. Indicate the information published per message, the corresponding topic, and the QoS level associated with each. Justify the sequence of events presented in the log.

To publish, subscribe to, and view messages on MQTT topics, we will need a support tool. There are several solutions, for example, MQTT Explorer and MQTXX. Due to the latter supporting different transport protocols (MQTT/TCP, MQTT/TLS, MQTT/WebSocket), offering an appealing graphical environment, and being cross-platform, we will opt to use it in this class.

Question 3: Install the MQTXX client and configure it as shown in Figure 2. **Important note:** because different clients may access the broker simultaneously, it is

recommended that you enter a unique Client ID, for example, by appending the student's number.

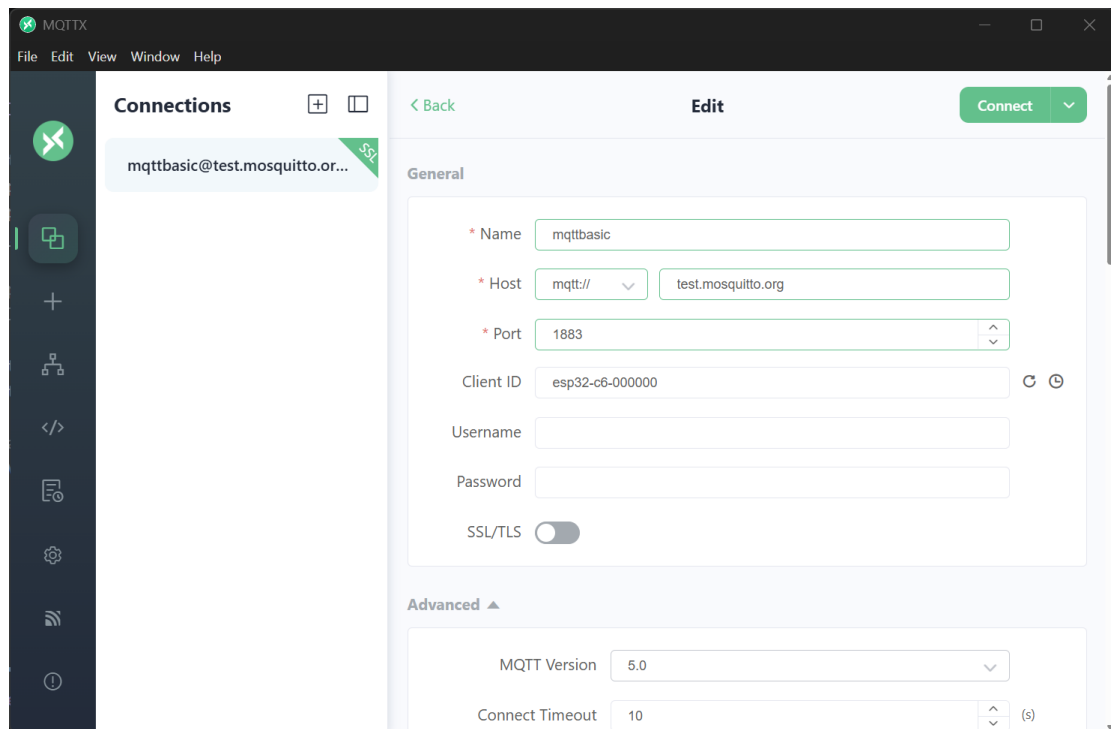


Figure 2 - MQTTX client configuration.

Start the client by subscribing to the `/topic/qos0` topic. If you restart the program on the board, you should observe a result in the MQTT client similar to that shown in Figure 3.

Question 4: Cause a momentary interruption in the AP, for example, by temporarily turning it off for 2/3 seconds and restoring it to operation. You should observe behavior consistent with the following listing:

```
I (40785) wifi:bcn_timeout,ap_probe_send_start
I (43285) wifi:ap_probe_send over, reset wifi status to disassoc
...
I (43305) example_connect: Wi-Fi disconnected 200, trying to reconnect...
E (43315) transport_base: tcp_read error, errno=Software caused connection abort
E (43325) mqtt_client: esp_mqtt_handle_transport_read_error: transport_read() error: errno=113
I (43325) mqtt_example: MQTT_EVENT_ERROR
E (43335) mqtt_example: Last error captured as transports socket errno: 0x71
I (43345) mqtt_example: Last errno string (Software caused connection abort)
E (43345) mqtt_client: mqtt_process_receive: mqtt_message_receive() returned -2
I (43355) mqtt_example: MQTT_EVENT_DISCONNECTED
I (45725) example_connect: Wi-Fi disconnected 201, trying to reconnect...
I (48125) example_connect: Wi-Fi disconnected 201, trying to reconnect...
```

```

...
I (52095) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (52095) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (52095) example_connect: - IPv4 address: 172.20.10.2,
...
I (58655) mqtt_example: MQTT_EVENT_CONNECTED
I (58655) mqtt_example: sent publish successful, msg_id=20071
I (58655) mqtt_example: sent subscribe successful, msg_id=8387
I (58665) mqtt_example: sent subscribe successful, msg_id=1962
I (58665) mqtt_example: sent unsubscribe successful, msg_id=14799
I (58755) mqtt_example: MQTT_EVENT_PUBLISHED, msg_id=20071
I (58865) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=8387
I (58865) mqtt_example: sent publish successful, msg_id=0
I (58865) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=1962
I (58865) mqtt_example: sent publish successful, msg_id=0
I (58875) mqtt_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=14799
I (59065) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (59175) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data

```

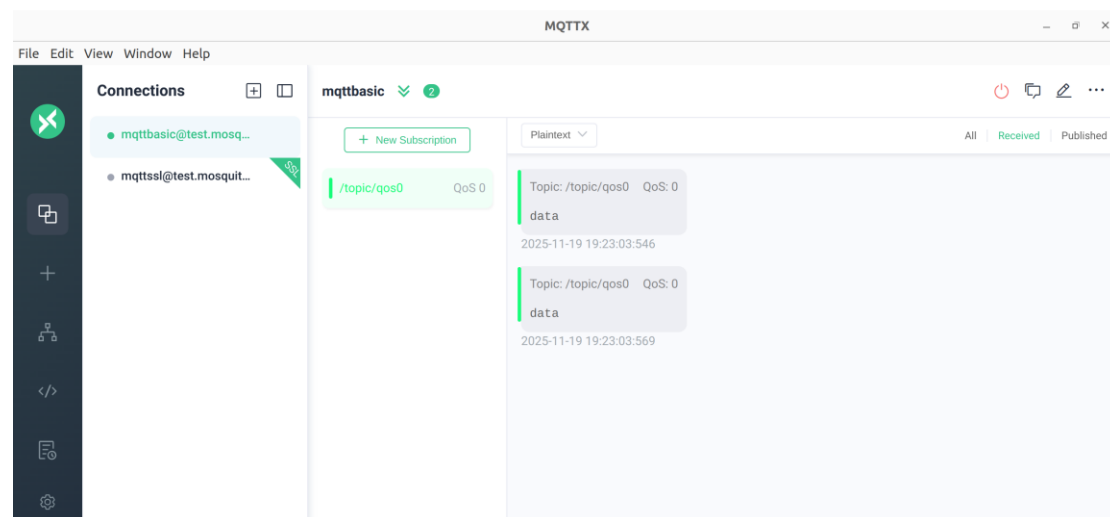


Figure 3 - MQTTX client configuration.

As you can see, the connection was re-established and the IP address was obtained again. The MQTT messages are published again.

Question 5: When the MQTT client initializes, it also subscribes to the `/topic/qos0` topic. Therefore, it also receives the messages that are published there. Using the MQTTX client, post a message to this topic with the content "ASE" and observe the

Monitor terminal confirming that the corresponding data event (`MQTT_EVENT_DATA`) is launched.

3.2 MQTT over TLS (mqtt/ssl)

The MQTT over SSL example included in the ESP-IDF platform demonstrates how to establish secure communication between the ESP32-C6 and an MQTT broker via the Transport Layer Security (TLS) protocol, using the standard port 8883. In this example, the device connects to the broker using the URI `mqtt://`, and the firmware incorporates a Certificate Authority (CA) certificate to validate the remote server's identity, ensuring the confidentiality and integrity of the exchanged data.

As in the simple MQTT example, the code shows how to configure the client, connect to the broker, publish messages, subscribe to topics, and handle events (such as message receipts or connection failures). The main difference is in the TLS security layer, which requires additional configuration, such as including the CA certificate in the project.

Question 6: In VSCode, create a new project from the *protocols -> mqtt -> ssl* template. Next, modify the *CMakeLists.txt* file at the project root to include a PEM certificate in the final ESP32 application firmware. It should reference the binary generated by the project and not the default value, as shown below:

```
target_add_binary_data(${PROJECT_NAME}.elf "main/mqtt_eclipseprojects_io.pem" TEXT)
```

During the build process, the contents of the PEM file are converted into a byte array and automatically included in the firmware. It is then accessible in the code through automatically generated external variables.

Only after making this change should you proceed to use **menuconfig** to set the SSID and Password of the Wi-Fi network you configured on your mobile phone. Access to this configuration in the project is found in the Example Connection Configuration section of **menuconfig**. Additionally, set the Broker URI to *mqtt://test.mosquitto.org:8883*. This configuration is available in the Example Configuration section of **menuconfig**.

Important note: Since you are using a different broker than the one contemplated in the original exercise, you will need to download the corresponding Certificate Authority (CA) file from the *test.mosquitto.org* website and replace it in the project's main folder before you can proceed with compilation.

Then compile the program and download the firmware to the SoC. You should observe the following event log:

```
I (292) cpu_start: Unicore app
I (300) cpu_start: Pro cpu start user code
...
I (406) main_task: Started on CPU0
I (406) main_task: Calling app_main()
I (406) mqtt_example: [APP] Startup..
...
I (3236) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = ae:5a:9d:a1:16:45
...
I (4396) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (4396) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
...
I (4426) example_common: - IPv4 address: 172.20.10.2,
I (4436) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
ESP_IP6_ADDR_IS_LINK_LOCAL
I (4446) mqtt_example: [APP] Free memory: 318512 bytes
...
I (4456) main_task: Returned from app_main()
...
I (5916) mqtt_example: MQTT_EVENT_CONNECTED
I (5916) mqtt_example: sent subscribe successful, msg_id=22264
I (5916) mqtt_example: sent subscribe successful, msg_id=21810
I (5916) mqtt_example: sent unsubscribe successful, msg_id=42298
I (6016) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=22264
I (6016) mqtt_example: sent publish successful, msg_id=0
I (6116) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=21810
I (6116) mqtt_example: sent publish successful, msg_id=0
I (6116) mqtt_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=42298
I (6216) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (6326) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
```

Question 7: Explore the code and justify the sequence of MQTT events presented in the Monitor console.

Question 8: Starting from Figure 4, configure an SSL/TLS session in the MQTTX client according to the parameters presented. Note that you must include the CA certificate in the box “CA File”. As for the previous MQTTX client configuration, select a unique Client ID, for example, by appending the student’s number.

The download of the CA certificate that signs the Mosquitto test broker at test.mosquitto.org:8883 can be done as follows:

```
wget https://test.mosquitto.org/ssl/mosquitto.org.crt -O mosquitto_org.crt
```

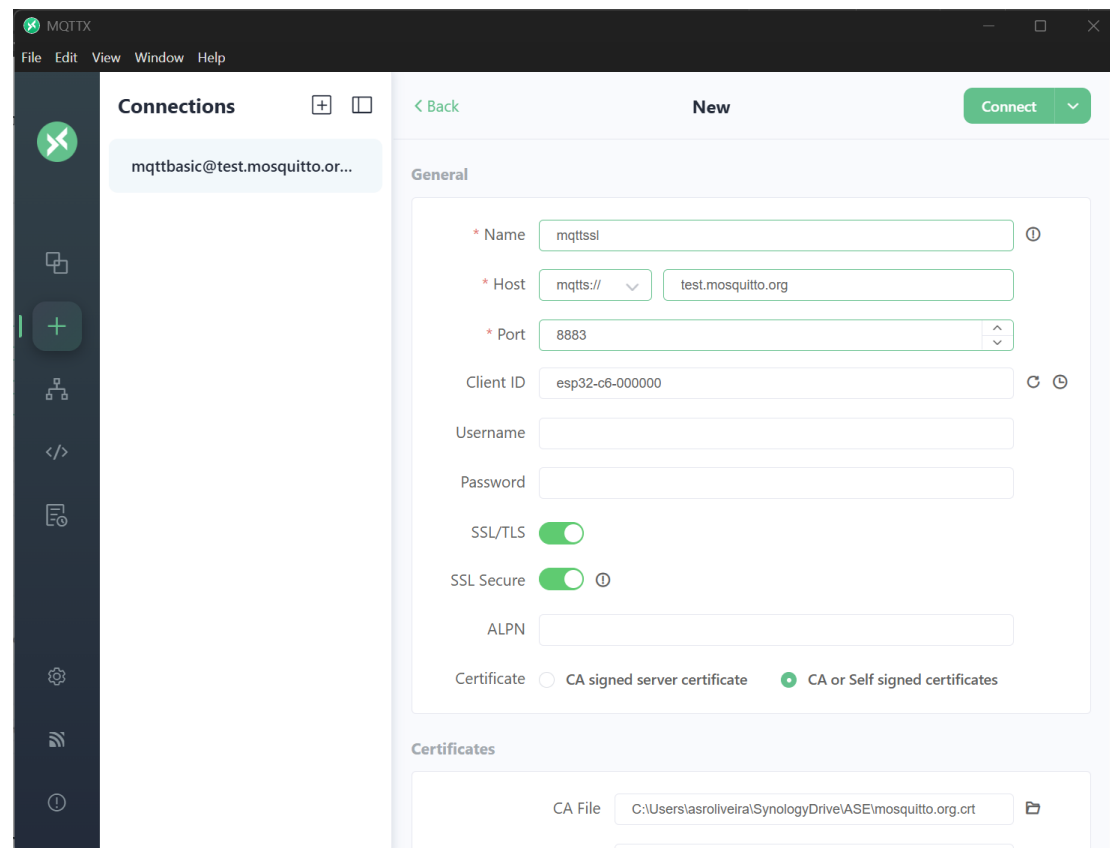


Figure 4 - MQTTX configuration to operate in MQTT over TLS mode (mqtt/ssl).

If you restart the board, you will see information about publishing data on the topic */topic/qos0* in the MQTTX client console, as illustrated in Figure 5.

Question 9: As with the MQTT over TCP publishing scenario, the ESP32-C6 application also subscribes to the topic `/topic/qos0`. Using the MQTTX client, post the information *one two three* to this topic and observe the resulting data event (MQTT_EVENT_DATA) in the ESP32 Terminal Monitor.

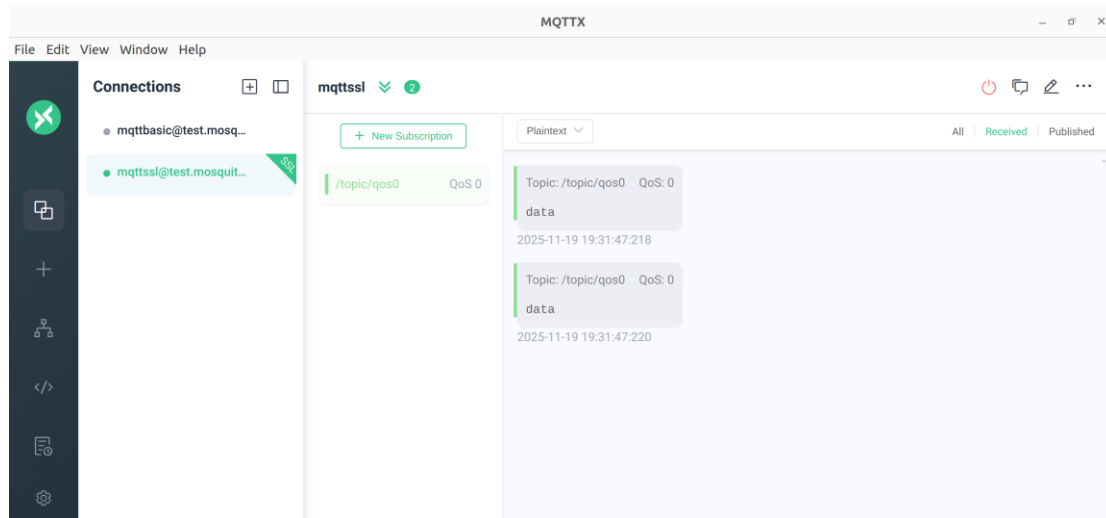


Figure 5 - Monitoring of the `/topic/qos0` topic in MQTT mode over TLS (mqtt/ssl).

3.3 Broker Mosquitto

In this part of the script, you will need to install the *Mosquitto* broker and configure it to operate in both MQTT over TCP and MQTT over TLS modes, making it compatible with the examples explored previously.

Important note: The Mosquitto broker must be installed on a personal laptop with direct connectivity to the ESP32-C6 board via the local Access Point (cell phone), i.e., they must be on the same network. Given that the Common Name (CN) of the CA certificate must be equal to the broker's IP address (so that TLS authentication does not fail), it is recommended that the laptop's connection to the AP be made at the beginning and that the respective IP address assigned by the DNS server running on the AP be registered. This will save a lot of work later.

The installation is simple, just run the following commands in the command line:

```
sudo apt-get update

sudo apt-get install mosquitto mosquitto-clients
```

Next, generate the certificate associated with the broker. Note that you must replace the IP address in the example with the laptop's IP address where the broker will run.

```
cd /etc/mosquitto/certs

# Replace 192.168.1.50 with the actual IP of your PC in the LAN

sudo openssl req -new -x509 -days 365 -nodes \
-out mosquitto.crt \
-keyout mosquitto.key \
-subj "/CN=192.168.1.50"

sudo chown mosquitto:mosquitto mosquitto.*

sudo chmod 600 mosquitto.key

sudo chmod 644 mosquitto.crt
```

Next, edit the file `/etc/mosquitto/mosquitto.conf` so that it contains the following:

```
pid_file /run/mosquitto/mosquitto.pid

user mosquitto

persistence true

persistence_location /var/lib/mosquitto/

log_dest syslog

log_dest stdout

log_type error warning notice information

include_dir /etc/mosquitto/conf.d
```

Create the file `/etc/mosquitto/conf.d/local.conf` with the following content:

```
# Plain MQTT, no auth

listener 1883

allow_anonymous true

# TLS MQTT, no auth

listener 8883

cafile /etc/mosquitto/certs/mosquitto.crt
certfile /etc/mosquitto/certs/mosquitto.crt
keyfile /etc/mosquitto/certs/mosquitto.key

allow_anonymous true
```

Next, launch the *Mosquitto* broker:

```
sudo systemctl start mosquitto
```

To test if this is working correctly, try the following commands. Note: the IP address with the corresponding address for your broker must be replaced.

```
# Plain

mosquitto_sub -h 192.168.1.50 -p 1883 -t test -v

# TLS

mosquitto_sub -h 192.168.1.50 -p 8883 \
--cafile /etc/mosquitto/certs/mosquitto.crt \
-t test -v
```

After performing these operations, you will have validated that the local *Mosquitto* broker is functioning correctly.

Question 10: Repeat all the steps above, but instead of using the remote *Mosquitto* broker, use the local *Mosquitto* Broker. Important Note: In the application's source code that uses TLS, you must make the necessary changes so that the auto-certificate CA generated in this installation process is used, and not the original CA certificate.